

Kubernetes and Cloud Native Associate - KCNA

Container Orchestration Security

Role Based Access Controls

In this guide, we explore Role-Based Access Controls (RBAC) in Kubernetes, including how to create and manage roles and role bindings effectively. RBAC provides a robust mechanism for managing access to resources within a Kubernetes cluster.

Creating a Role

A Role in Kubernetes encapsulates a set of permissions for resources within a namespace. To create a role, define a role object in a YAML file with the following essential components:

- Set the API version to `rbac.authorization.k8s.io/v1`
- Specify the `kind` as `Role`
- Provide a metadata name (for example, "developer")
- List the rules that define the API groups, resources, and permitted verbs (actions)

For resources within the core group, leave the API groups field blank. For resources in other groups, specify the group name explicitly.

Below is an example YAML configuration that creates a "developer" role. This role allows actions such as listing, getting, creating, updating, and deleting pods, and includes an extra rule to create ConfigMaps:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["list", "get", "create", "update", "delete"]
- apiGroups: [""]
  resources: ["ConfigMap"]
  verbs: ["create"]
```

Create the role using the following command:

```
kubectl create -f <role-definition.yaml>
```



Note

Both roles and role bindings in Kubernetes are namespace-scoped. In this example, the `developer` role applies only to the default namespace. To apply a role in another namespace, add the `namespace` field within the metadata section of your YAML file.

Creating a Role Binding

After defining a role, bind it to a specific user using a RoleBinding. A role binding connects a user to a role within a namespace, thereby granting the user the permissions defined in that role.

Below is an example YAML configuration that creates a role binding named "devuser-developer-binding". This binding assigns the "developer" role to the user `dev-user`:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: devuser-developer-binding
subjects:
- kind: User
  name: dev-user
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: developer
  apiGroup: rbac.authorization.k8s.io
```

Create the role binding using the command:

```
kubectl create -f <rolebinding-definition.yaml>
```

Viewing and Verifying Roles and Bindings

Listing Roles and Role Bindings

You can verify that your roles and role bindings have been created correctly using the `kubectl` commands below:

```
kubectl get roles
```

This command outputs a list of roles, for example:

```
NAME      AGE
developer 4s
```

Similarly, list the role bindings:

```
kubectl get rolebindings
```

Output example:

```
NAME                               AGE
devuser-developer-binding         24s
```

Describing Specific Resources

To get detailed information about a specific role, use:

```
kubectl describe role developer
```

Example output:

```
Name:                developer
Labels:              <none>
Annotations:         <none>
PolicyRule:
  Resources            Non-Resource URLs  Resource Names  Verbs
  -----
  ConfigMap            []                  []              [create]
  pods                 []                  []              [get watch list create delete]
```

Similarly, for a detailed view of a role binding, run:

```
kubectl describe rolebinding devuser-developer-binding
```

Example output:

```
Name:                devuser-developer-binding
Labels:              <none>
```

```
Annotations:      <none>
Role:
  Kind:           Role
  Name:           developer
Subjects:
  Kind  Name      Namespace
  ----  ---      -
  User  dev-user
```

Verifying User Permissions

Use the `kubectl auth can-i` command to verify if a user has permission to perform a specific action on a resource. For instance, to check if you can create deployments or delete nodes, execute:

```
kubectl auth can-i create deployments
kubectl auth can-i delete nodes
# Expected output: no
```

If you are an administrator wishing to test permissions for another user without switching accounts, use the `--as` flag. For example, to check if `dev-user` can create deployments or pods, run:

```
kubectl auth can-i create deployments --as dev-user
kubectl auth can-i create pods --as dev-user
# Expected output: yes
```

Remember, you can also include the `--namespace` flag to specify the namespace for these commands.

Restricting Access to Specific Resources

Sometimes it's necessary to restrict a role's permissions to only specific resource instances. For example, if a user should only have access to pods named "blue" and "orange", you can utilize the `resourceNames` field in the role's rule:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "create", "update"]
  resourceNames: ["blue", "orange"]
```

This configuration ensures that the user can only interact with the specified pods.

Conclusion

This guide covered the essentials of Kubernetes Role-Based Access Controls, including creating roles, binding them to users, and verifying permissions. For more detailed information on Kubernetes RBAC, refer to the official [Kubernetes Documentation](#).

Happy securing your Kubernetes clusters!



Watch Video

Watch video content



Beta



🔍 Search docs...



Kubernetes and Cloud Native Associate - KCNA ▼